OXFORDABLE CAREERS

WEEK 3 OF 3 · PARTICIPANT WORKBOOK v2.0

Build, Test, Deploy & Demo

User Interface, AI, Testing and Release

Name: _____

Cohort: _____

Date: _____

My GitHub repository: _____

Salesforce Developer Introductory Programme · Labs 5, 6, 7 and 8 · UAT and demo · **Version 2.0 — Extension Pack update + complete AI Build Prompt series**

This is the workbook you keep. It contains your acceptance test script, your deployment commands and your portfolio write-up.

Welcome to Week 3

Today you finish the application, prove it works, ship it to another org, and demonstrate it. By the end of this session you have something real to show an employer.

> **WHAT CHANGED IN VERSION 2.0 OF THIS WORKBOOK**
>
> Week 3 is no longer the end. The same project continues into the **CareerLaunch Extension Pack** — invoicing and payments, certificate PDFs (filed *and* emailed), org-wide error logging, multi-currency and multi-language, and an Experience Cloud student portal with automatic login provisioning — documented in the **Complete Build Guide**, the Extension Workbooks (Parts A–C) and the E2E Test Document. Appendix E of this workbook carries the full AI Build Prompt series (P1–P12) that can reproduce everything, in order, including seed data.

Before you arrive — checklist

- ☐ My org contains Course, Cohort and Enrolment with all the fields, the roll-up, both formulas and both validation rules.
- ☐ `EnrolmentTrigger` and `EnrolmentTriggerHandler` are deployed and active.
- ☐ `StudentSummaryQueueable` and `CohortAvailabilityService` are saved without compile errors.
- ☐ The record-triggered flow `Enrolment_Confirmed_Actions` is **activated**.
- ☐ I have sample data: at least three Contacts **with email addresses**, and two open Cohorts with seats free.
- ☐ I attempted Homework 2 task 5 — a test method, however rough.
- ☐ VS Code, Salesforce CLI and Git all work on my machine.

> **ANYTHING NOT TICKED?**
>
> Post in the group **before** the session. Today has four labs back to back and we cannot rebuild Week 2 in the room. If your Apex will not compile, send the error message — it is almost always a field API name spelling.

What today covers

TIME	BLOCK	WHAT YOU WILL DO
0:00–0:10	Recap & homework review	We look at two of your test attempts on screen
0:10–0:50	**Lab 5 — Lightning Web Components**	Apex controllers, two LWCs, the Enrolment Console page
0:50–1:20	**Lab 6 — Agentforce agent**	Actions, agent, topic, instructions, testing in preview
1:20–1:30	Break	—
1:30–2:00	**Lab 7 — Testing & debugging**	Test data factory, test classes, coverage, debug logs
2:00–2:30	**Lab 8 — Security & deployment**	Permission set, CLI deployment to a second org, Git
2:30–2:50	UAT & demo round	Run the acceptance script; two-minute demos
2:50–3:00	PD1 map, career support, close	Where this leads next

The three stories we deliver today

ID	USER STORY	ACCEPTANCE CRITERIA

US-10	As an admissions officer, I want a single console showing open cohorts with seats remaining and an enrol button, so that I can work from one screen.	A Lightning page lists open cohorts, filterable by course, showing seats remaining; selecting one allows enrolling a student and shows a success toast.
US-11	As a prospective student, I want to ask an assistant which cohorts have seats available, so that I get an instant answer.	An Agentforce agent answers “which Advanced cohorts have seats?” using live org data.
US-12	As a student, I want to ask the assistant about the status of my enrolment, so that I do not have to email the team.	The agent returns the student's cohort, start date and enrolment status when given their email address.
US-14	As the delivery team, I want the solution covered by automated tests and deployable to another org, so that we can release safely.	75% Apex coverage or better with meaningful assertions; a documented CLI deployment succeeds against a clean org.

Lab 5 — Lightning Web Components

LAB 5 · 40 MINUTES · HANDS-ON

Lightning Web Components

Build the Enrolment Console: two custom components, two Apex controllers, one Lightning page.

The concept, in one paragraph. A Lightning Web Component is **three files**: an HTML template (what it looks like), a JavaScript class (what it does), and a metadata XML file (where it is allowed to be used). Data comes from Apex in one of two ways — through the **wire service** (reactive, cacheable, for reading) or **imperatively** (on demand, for writing).

Build order

1. **The Apex controllers.** Create `CohortController` and `EnrolmentController` (Appendix A.1 and A.2). `@AuraEnabled(cacheable=true)` for the reads — this is what makes `@wire` possible. Plain `@AuraEnabled` for the write: a cacheable method is not allowed to perform DML.
2. `cohortExplorer` (Appendix B.1) — a combobox of courses and a `lightning-datatable` of open cohorts, populated by the wire adapter. Selecting a row dispatches a `cohortselected` custom event.
3. `enrolmentPanel` (Appendix B.2) — receives the selected cohort through `@api`, shows a contact picker, calls Apex imperatively, fires a toast, navigates to the new record, and tells the parent to refresh.
4. `enrolmentConsole` (Appendix B.3) — the small wrapper that listens for `cohortselected`, passes the record down, and refreshes the explorer when an enrolment is created.
5. **The Lightning page.** Setup → Lightning App Builder → New → App Page → two-column layout. Drop `enrolmentConsole` on, name it **Enrolment Console**, then Save → **Activate** → in the activation dialog choose *App, Record Type, and Profile* and add it to your app's navigation if offered.
6. **Create the Tab — do not skip this.** Activating an App Page does **not** put it in the App Launcher; that needs an explicit Tab pointing at it. Setup → Quick Find `Tabs` → find the **Lightning Page Tabs** section → **New** → select page **Enrolment Console** → pick a tab style/icon → Next → Next (profile visibility can be skipped; permission sets handle it) → Save. Without this step the page exists but has no address the App Launcher can search for.

Teaching beats — the things to actually remember

BEAT	THE POINT
`@wire` versus imperative	Reads versus writes. Wired data is cached and refreshes reactively; `refreshApex` forces a reload.
`@api`	A public property — how a parent passes data *down* to a child.
`CustomEvent`	How a child talks *up*. Memorise: **data down, events up**.
`ShowToastEvent` / `NavigationMixin`	Standard user feedback / navigation.
try/catch + `AuraHandledException`	NFR-04 in practice: users see a sentence, not a stack trace.

CHECKPOINT — THE WHOLE ARCHITECTURE PROVEN IN ONE ACTION

Enrol a student from the console: a **toast** appears, the datatable's **seat count drops**, and an attempted overbook is **still blocked by the Week 2 trigger** — your message, surfaced through the new UI. The console is just another channel on the same protected service layer. *(In the Extension Pack the same principle carries a whole student portal — see Appendix E, prompt P10.)*

Lab 6 — Agentforce Agent

LAB 6 · 30 MINUTES · HANDS-ON

Agentforce Agent

Give the Apex you wrote in Week 2 to an AI agent, and watch it answer real questions from live org data.

The concept, in one paragraph. An agent is made of **topics** (what it can help with), **instructions** (how it should behave), and **actions** (what it can actually do). Your `CohortAvailabilityService` from Lab 4 becomes an agent action **with no rewriting whatsoever**.

Prerequisites — check these first

- ☐ Agentforce / Einstein Setup is enabled (Setup → Einstein Setup → turn on).
- ☐ `CohortAvailabilityService` and `EnrolmentStatusService` are deployed and compile.
- ☐ Sample Courses and Cohorts exist with genuinely available seats.
- ☐ My user has the required Agentforce permission set assigned.

> **IF AGENTFORCE IS UNAVAILABLE IN YOUR ORG**
> Some Developer Edition orgs do not offer it. That is not a failure on your part: the instructor demonstrates from the reference org, you follow the build sheet as written homework, and the learning objective survives — the important idea is the `@InvocableMethod` bridge, and you already own that.

Build sheet

1. **Create the second action class.** `EnrolmentStatusService` (Appendix A.3) returns a student's enrolment status from an email address.
2. **Create the agent.** Setup → Agentforce → Agents → New Agent. Name `Oxfordable Student Assistant`; type Employee/internal; role “Answers questions about our courses, cohort availability and a student's own enrolment status”; company “Oxfordable Careers, a technology training provider”; tone “Warm, concise, professional. Never invent dates, prices or seat numbers.”
3. **Create two agent actions** from your invocables: **Find Available Cohorts** (input Course Name; outputs Answer, Available Cohorts, Seats Available) and **Check Enrolment Status** (input Student Email; outputs Answer, Found).
4. **Write the descriptions properly.** Every input and output needs a clear description — the agent reads these to decide what to pass and how to answer. Write them **for a colleague who has never seen your org**: thin descriptions are the number one reason agents misbehave.
5. **Create the topic** `Course Enrolment Enquiries`. Classification: “Use this topic when someone asks about our courses, when a cohort starts, whether seats are available, how to join, or the status of an enrolment they already hold.” Scope: answer only from the assigned actions; never speculate. Instructions: ask which course if unclear; always state start date and seats; offer the next cohort if none; ask for the booking email for status questions; never share one student's details with another; pass anything else to the team. Assign both actions.
6. **Test in the conversation preview** — the five scripted questions in the UAT table (availability, start dates, status flow, out-of-scope), then **read the reasoning trace** after each: which topic fired, which action ran. An agent that answers badly is nearly always choosing the wrong action, and the trace tells you so.
7. **Activate** the agent.

THE POINT OF THIS LAB

The agent is not magic. It is a language model given **your** topic definition and **your** Apex. Everything it can truthfully say about seat availability comes from the SOQL query you wrote last week. That is the honest, sellable version of “I have built with AI”.

Lab 7 — Testing and Debugging

LAB 7 · 30 MINUTES · HANDS-ON

Testing and Debugging

Prove the capacity rule works — and that it will keep working after somebody else edits your code.

WHY WE TEST

Salesforce will not let you deploy Apex to production below **75% coverage**. But coverage is the floor, not the goal. Tests exist so the next person to change this code finds out *immediately* if they broke your capacity rule. That person is often you, three months later.

Build

1. `TestDataFactory` (Appendix A.4) — reusable creation of a course, a cohort, contacts and enrolments. Two rules while we build it: hard-coded record IDs are forbidden, and `SeeAllData=false` (the default) means every test creates its own world.
2. `EnrolmentTriggerTest` (Appendix A.5) — five methods: defaults (US-02) · overbooking blocked, a negative test using `Database.insert(record, false)` (US-04) · 200-record bulk (NFR-01) · duplicate blocked (US-05) · the queueable, forced to run by `Test.startTest()/stopTest()` (US-13).
3. `CohortServicesTest` (Appendix A.6) — covers both controllers and both invocable services, including the error paths.

```
sf apex run test --target-org devorg --code-coverage --result-format human --wait 10
```

Our test standards

1. One test class per class under test, named `<ClassName>Test`.
2. All data created inside the test — never rely on org data.
3. Shared setup via `TestDataFactory` and `@testSetup`.
4. Every method contains at least one meaningful `Assert`.
5. Test the **unhappy** path, not just the happy one.
6. Bulk test with 200 records for anything a trigger touches.
7. Coverage floor 75%; our project target is 90%.

Debugging toolkit

SYMPTOM	FIRST PLACE TO LOOK
“An unexpected error occurred” in the UI	Debug Logs → most recent → search `FATAL_ERROR`
Apex not firing	Is the trigger active? Does the record meet the entry criteria?
Flow not firing	Not activated, or wrong entry conditions. Check Paused and Failed Flow Interviews.
LWC renders blank	Browser console (F12); the `.js-meta.xml` targets; Apex class access on the permission set

Agent gives a vague answer	The reasoning trace — wrong topic, or thin action descriptions
Test fails only in bulk	SOQL or DML inside a loop

TARGET

☐ A green test run, with overall code coverage of **75% or higher**. Write your number here: _____ %

Lab 8 — Security and Deployment

LAB 8 · 30 MINUTES · HANDS-ON

Security and Deployment

Grant access properly, get your source onto your machine and into GitHub, and deploy the whole application to a second org.

Step 1 — Permission set (NFR-05)

Setup → Permission Sets → New → label `Enrolment Console User`. Object Settings: Course, Cohort, Enrolment → Read, Create, Edit; Contact → Read. Field permissions for all custom fields. Apex Class Access: CohortController, EnrolmentController, CohortAvailabilityService, EnrolmentStatusService. Tab settings: Courses, Cohorts, Enrolments, **Enrolment Console** → Visible (the Console tab from Lab 5 needs this too, or it never shows for anyone but you). Assign it to yourself, and to a second test user if you have one.

> **WHY NOT JUST EDIT THE PROFILE?**
>
> Profiles are shared by many users and painful to deploy and audit. Permission sets are additive, granular and portable. Editing the System Administrator profile to “make it work” is the classic beginner mistake, and explicitly against NFR-05.

Step 2 — Get the source onto your machine

```
sf org login web --alias devorg
sf project generate --name careerlaunch
cd careerlaunch
sf project retrieve start --manifest manifest/package.xml --target-org devorg
```

Use the `package.xml` in Appendix C. Everything you built with clicks is now text on disk.

Step 3 — Version control

```
git init
git add .
git commit -m "CareerLaunch v1: data model, automation, Apex, LWC, agent"
git branch -M main
git remote add origin https://github.com/<username>/careerlaunch.git
git push -u origin main
```

> **THIS REPOSITORY IS A PORTFOLIO ASSET**
>
> It goes on your CV and in your LinkedIn Featured section. Write the repository URL on the cover of this workbook now, before you forget it.

Step 4 — Deploy to a second org

```
sf org login web --alias targetorg
# Validate first - deploys nothing, but runs every check
sf project deploy start --manifest manifest/package.xml --target-org targetorg \
    --test-level RunLocalTests --dry-run
# Then deploy for real
sf project deploy start --manifest manifest/package.xml --target-org targetorg \
    --test-level RunLocalTests
```

Step 5 — Post-deployment checklist

#	ACTION — DEPLOYMENT IS NEVER FINISHED AT "THE DEPLOY SUCCEEDED"
1	Confirm all Flows are **Active** in the target org — they can deploy inactive
2	Assign the `Enrolment Console User` permission set to test users (fresh deploys grant no FLS)
3	Add the Enrolment Console page and the tabs to the app navigation
4	Activate the Agentforce agent and re-test one question
5	Check org-wide email deliverability is **All Email**
6	Load reference data (Courses, Cohorts) — **records are not deployed**
7	Run smoke tests UAT-03, UAT-04 and UAT-10 in the target org
8	Tag the release: `git tag v1.0 && git push --tags`

DISCUSSION — BE READY WITH AN ANSWER

It is Friday afternoon. The deployment succeeded, and the capacity rule is now blocking every save. **What do you do?** Think about: deactivating the trigger, reverting the Git commit and redeploying, and — best of all — why the validate-only run in Step 4 exists at all.

User Acceptance Testing

Run this script against your own org and record the result. It is the evidence that your application does what the client asked for — and a genuinely impressive thing to show at interview. *(The full platform, including the Extension Pack, has its own 46-case script: the E2E Test Document.)*

ID	STORY	TEST STEP	EXPECTED RESULT	✓
UAT-01	US-01	Create a Course and a Cohort linked to it	Both save; the Cohort shows the Course	☐
UAT-02	US-01	Set the Cohort End Date before the Start Date	Validation error shown	☐
UAT-03	US-03	Create one enrolment on a cohort with capacity 3	Enrolled Count = 1, Seats Remaining = 2	☐
UAT-04	US-04	Enrol a fourth student on a capacity-3 cohort	Save blocked with the custom capacity message	☐
UAT-05	US-05	Enrol the same student on the same cohort twice	Save blocked (duplicate key)	☐
UAT-06	US-06	Change an enrolment status to Confirmed	Email sent to the student; follow-up Task created	☐
UAT-07	US-07	Fill the final seat on a cohort	Cohort Status changes to Full automatically	☐
UAT-08	US-08	Run the Quick Enrol screen flow end to end	Enrolment created; confirmation shows the record number	☐
UAT-09	US-09	Set a cohort start date to today + 3 and run the scheduled flow	Reminder generated for confirmed enrolments only	☐
UAT-10	US-10	Open the Enrolment Console, filter, select a cohort, enrol a student	Toast shown; seat count refreshes; record created	☐
UAT-11	US-11	Ask the agent “which Advanced cohorts have seats?”	Live cohorts with dates and seats remaining	☐
UAT-12	US-12	Ask the agent for enrolment status with a known email	Cohort, start date and status returned	☐
UAT-13	US-13	Check the student's Contact record	Total Enrolments reflects active enrolments	☐
UAT-14	US-14	Run all tests	All pass; overall coverage 75% or higher	☐
UAT-15	US-14	Deploy to a second org; repeat UAT-03 and UAT-10 there	Both behave identically	☐

Your two-minute demo

SECONDS	COVER	SAY SOMETHING LIKE

0–20	The problem	“Oxfordable ran cohorts in spreadsheets and could not answer how many seats were left.”
20–50	The data model	“Three objects. Master-detail from Enrolment to Cohort so I could roll up the seat count.”
50–1:20	One automation	“The capacity rule is an Apex trigger, because a validation rule cannot count sibling records.”
1:20–1:45	The console	Enrol somebody live; show the toast and the seat count updating.
1:45–2:00	The agent	Ask it a question and show the answer came from your own SOQL.

RECORD IT

A two-minute video of your own working application is one of the strongest things a career changer can attach to a job application.

Completion, Portfolio & What Comes Next

Completion criteria

#	EVIDENCE	✓
1	A Developer org containing the CareerLaunch data model with sample data	☐
2	At least two working Flows — one record-triggered, one screen	☐
3	An active `EnrolmentTrigger` that blocks overbooking and duplicates	☐
4	A working Enrolment Console page built from your own LWCs	☐
5	An Agentforce agent that answers an availability question from live data	☐
6	A passing test run with 75% coverage or better	☐
7	A completed UAT script	☐
8	A public GitHub repository containing the source	☐
9	A two-minute recorded or live demo	☐

Meeting criteria 1 to 7 earns the Oxfordable Careers Certificate of Completion — a statement of attendance and demonstrated practical work, not a Salesforce credential. **PD1 is the credential that counts**, and it is what the Advanced Course prepares you for.

Write it up as a portfolio piece

CAREERLAUNCH — SALESFORCE COHORT ENROLMENT & STUDENT SUCCESS APP

Designed and built a Salesforce application managing course cohorts and student enrolments. Modelled the schema (custom objects, master-detail and lookup relationships, roll-up summaries). Automated confirmation and reminder processes with record-triggered, screen and scheduled Flows. Wrote bulkified Apex triggers with a handler pattern to enforce seat capacity and prevent duplicate enrolments, plus a Queueable job for asynchronous aggregation. Built a two-component Lightning Web Component console with wire and imperative Apex. Exposed Apex as invocable actions powering an Agentforce agent answering availability questions from live data. Achieved 90% Apex coverage and deployed via Salesforce CLI with a validated, test-gated release. *Extension: payment layer behind an interface + custom-metadata factory (stub and production HTTP client with named credential and idempotency key), automatic invoicing with frozen exchange rates, PDF certificates generated asynchronously and emailed to students, a rollback-proof platform-event error-logging framework, custom-label internationalisation (EN/FR), and an Experience Cloud portal with contact-scoped self-service payments and automatic login provisioning.* (GitHub link)

Ten interview questions this project prepares you to answer

1. When would you use a Flow instead of an Apex trigger?
2. What is the difference between a lookup and a master-detail relationship?
3. What does it mean to “bulkify” Apex, and how do you do it?
4. How would you prevent duplicate records without writing complex code?
5. What is a governor limit? Name three.
6. Before-save versus after-save flows — what is the difference and why does it matter?
7. How do you pass data from a parent LWC to a child, and back again?

8. Why is 75% coverage not the same as good testing?
9. Walk me through how you would deploy this to production safely.
10. How did you expose your Apex logic to an AI agent?

Where this leads

STEP	WHAT	WHEN
1	**The Extension Pack** — the same project, professional depth: payments & invoicing, certificates by portal and email, error logging, i18n, the student portal. Extension Workbooks A–C + the Complete Build Guide + the E2E Test Document.	The weeks after this session
2	**The Advanced Salesforce Developer Course** — trigger frameworks, integration patterns in anger, CI/CD, and PD1 exam preparation.	Reserve a place — WhatsApp +44 7448 717334 · +234 704 916 5925
3	**Career clinics** — CV, LinkedIn (this project in your Featured section), interview preparation.	Book after the programme

Score yourself again: go back to the skills map in your Week 1 workbook and fill in the Week 3 column. The difference is three weeks of work, and it is the honest measure of what you have gained.

Appendix A — Apex Source Code

USE THIS TO CATCH UP, NOT TO READ AHEAD

We type this code together in the labs. If you fall behind, copy from here and rejoin the group — do not sit stuck in silence. All classes use the British spelling `Enrolment__c`. Copy exactly. *(v2.0 note: this appendix is generated from the live repository, so it always matches the reference build — including `TestDataFactory.createStudent`, which the Extension Pack tests use.)*

A.1 CohortController — LWC read controller

```
public with sharing class CohortController {

    @AuraEnabled(cacheable=true)
    public static List<Course__c> getActiveCourses() {
        return [
            SELECT Id, Name, Course_Level__c, Fee__c
            FROM Course__c
            WHERE Active__c = true
            WITH SECURITY_ENFORCED
            ORDER BY Name ASC
            LIMIT 200
        ];
    }

    @AuraEnabled(cacheable=true)
    public static List<Cohort__c> getOpenCohorts(Id courseId) {
        String status = 'Open';
        List<Cohort__c> cohorts = (courseId == null)
            ? [SELECT Id, Name, Cohort_Label__c, Course__r.Name, Start_Date__c, Delivery_Mode__c,
                      Capacity__c, Enrolled_Count__c, Seats_Remaining__c, Status__c
               FROM Cohort__c
               WHERE Status__c = :status AND Start_Date__c >= TODAY
               WITH SECURITY_ENFORCED
               ORDER BY Start_Date__c ASC LIMIT 100]
            : [SELECT Id, Name, Cohort_Label__c, Course__r.Name, Start_Date__c, Delivery_Mode__c,
                      Capacity__c, Enrolled_Count__c, Seats_Remaining__c, Status__c
               FROM Cohort__c
               WHERE Status__c = :status AND Start_Date__c >= TODAY AND Course__c = :courseId
               WITH SECURITY_ENFORCED
               ORDER BY Start_Date__c ASC LIMIT 100];
        return cohorts;
    }
}
```

A.2 EnrolmentController — LWC write controller

```
public with sharing class EnrolmentController {

    @AuraEnabled
    public static Enrolment__c createEnrolment(Id cohortId, Id studentId, String status) {

        if (cohortId == null || studentId == null) {
            throw new AuraHandledException('Please choose both a cohort and a student.');
        }

        Enrolment__c enrolment = new Enrolment__c(
            Cohort__c  = cohortId,
            Student__c = studentId,
            Status__c  = String.isBlank(status) ? 'Confirmed' : status
        );
```

```
        try {
            insert enrolment;
        } catch (DmlException e) {
            // Surfaces the trigger message (capacity) or the unique-key message (duplicate)
            // as a readable sentence rather than a stack trace. See NFR-04.
            throw new AuraHandledException(friendlyMessage(e));
        }

        return [
            SELECT Id, Name, Status__c, Cohort__r.Cohort_Label__c, Student__r.Name
            FROM Enrolment__c
            WHERE Id = :enrolment.Id
            LIMIT 1
        ];
    }

    @TestVisible
    private static String friendlyMessage(DmlException e) {
        String raw = e.getDmlMessage(0);
        if (raw != null && raw.containsIgnoreCase('duplicate value found')) {
            return 'That student is already enrolled on this cohort.';
        }
        return String.isBlank(raw) ? e.getMessage() : raw;
    }
}
```

A.3 EnrolmentStatusService — second agent action (US-12)

```
public with sharing class EnrolmentStatusService {

    public class Request {
        @InvocableVariable(
            label='Student Email'
            description='The email address the student used when booking.'
            required=true)
        public String studentEmail;
    }

    public class Result {
        @InvocableVariable(label='Answer' description='A summary of the student current enrolments.')
        public String answer;

        @InvocableVariable(label='Found' description='True if any enrolment was found for that email.')
        public Boolean found;
    }

    @InvocableMethod(
        label='Check Enrolment Status'
        description='Returns the cohorts and enrolment statuses held by the student with the given email
address.'
        category='Oxfordable CareerLaunch')
    public static List<Result> checkStatus(List<Request> requests) {

        List<Result> results = new List<Result>();

        for (Request req : requests) {
            Result res   = new Result();
            String email = String.isBlank(req.studentEmail) ? '' : req.studentEmail.trim();

            List<Enrolment__c> enrolments = [
                SELECT Id, Name, Status__c, Enrolment_Date__c,
                       Cohort__r.Cohort_Label__c, Cohort__r.Start_Date__c,
                       Cohort__r.Delivery_Mode__c, Student__r.FirstName
                FROM Enrolment__c
```

```
                WHERE Student__r.Email = :email
                ORDER BY Cohort__r.Start_Date__c ASC
                LIMIT 10
            ];

            res.found = !enrolments.isEmpty();

            if (enrolments.isEmpty()) {
                res.answer = 'I could not find an enrolment for ' + email
                    + '. Please check the address used at booking, or I can pass this to the team.';
            } else {
                List<String> lines = new List<String>();
                for (Enrolment__c e : enrolments) {
                    lines.add(e.Cohort__r.Cohort_Label__c
                            + ' starting ' + e.Cohort__r.Start_Date__c.format()
                            + ' – status: ' + e.Status__c);
                }
                res.answer = 'Enrolments found: ' + String.join(lines, '; ') + '.';
            }
            results.add(res);
        }
        return results;
    }
}
```

A.4 TestDataFactory

```
@isTest
public class TestDataFactory {

    public static Course__c createCourse(String name, String level, Decimal fee) {
        Course__c course = new Course__c(
            Name              = name,
            Course_Level__c   = level,
            Duration_Weeks__c = 5,
            Fee__c            = fee,
            Active__c         = true
        );
        insert course;
        return course;
    }

    public static Cohort__c createCohort(Id courseId, Integer capacity, Integer daysFromToday) {
        Cohort__c cohort = new Cohort__c(
            Course__c        = courseId,
            Start_Date__c    = Date.today().addDays(daysFromToday),
            End_Date__c      = Date.today().addDays(daysFromToday + 35),
            Delivery_Mode__c = 'Online',
            Capacity__c      = capacity,
            Status__c        = 'Open'
        );
        insert cohort;
        return cohort;
    }

    public static List<Contact> createStudents(Integer howMany) {
        List<Contact> students = new List<Contact>();
        for (Integer i = 0; i < howMany; i++) {
            students.add(new Contact(
                FirstName = 'Test',
                LastName  = 'Student ' + i,
                Email     = 'student' + i + '@oxfordable.test'
            ));
        }
        // Bypass any active duplicate rules in the org so bulk tests are deterministic.
```

```
        Database.DMLOptions dml = new Database.DMLOptions();
        dml.DuplicateRuleHeader.allowSave = true;
        dml.optAllOrNone = true;
        Database.insert(students, dml);
        return students;
    }

    public static Contact createStudent(String email) {
        Contact student = new Contact(
            FirstName = 'Test',
            LastName  = 'Student',
            Email     = email
        );
        Database.DMLOptions dml = new Database.DMLOptions();
        dml.DuplicateRuleHeader.allowSave = true;
        Database.insert(student, dml);
        return student;
    }

    public static List<Enrolment__c> buildEnrolments(Id cohortId, List<Contact> students, String status)
{
        List<Enrolment__c> enrolments = new List<Enrolment__c>();
        for (Contact student : students) {
            enrolments.add(new Enrolment__c(
                Cohort__c  = cohortId,
                Student__c = student.Id,
                Status__c  = status
            ));
        }
        return enrolments;   // deliberately NOT inserted - the caller decides
    }
}
```

A.5 EnrolmentTriggerTest

```
@isTest
private class EnrolmentTriggerTest {

    @testSetup
    static void makeData() {
        Course__c course = TestDataFactory.createCourse(
            'Salesforce Developer Introductory', 'Introductory', 500);
        TestDataFactory.createCohort(course.Id, 3, 14);
        TestDataFactory.createStudents(5);
    }

    private static Cohort__c getCohort() {
        return [SELECT Id, Capacity__c, Enrolled_Count__c, Seats_Remaining__c FROM Cohort__c LIMIT 1];
    }

    @isTest
    static void testDefaultsApplied() {
        Cohort__c cohort = getCohort();
        Contact student  = [SELECT Id FROM Contact LIMIT 1];

        Enrolment__c enrolment = new Enrolment__c(Cohort__c = cohort.Id, Student__c = student.Id);

        Test.startTest();
        insert enrolment;
        Test.stopTest();

        Enrolment__c saved = [
            SELECT Status__c, Enrolment_Date__c, Amount_Paid__c, Enrolment_Key__c
            FROM Enrolment__c WHERE Id = :enrolment.Id
        ];
```

```
        Assert.areEqual('Applied', saved.Status__c, 'Status should default to Applied');
        Assert.areEqual(Date.today(), saved.Enrolment_Date__c, 'Enrolment date should default to
today');
        Assert.areEqual(0, saved.Amount_Paid__c, 'Amount paid should default to zero');
        Assert.isNotNull(saved.Enrolment_Key__c, 'Unique key should be populated');
    }

    @isTest
    static void testOverbookingBlocked() {
        Cohort__c cohort = getCohort();                          // capacity 3
        List<Contact> students = [SELECT Id FROM Contact LIMIT 4];

        List<Enrolment__c> firstThree = TestDataFactory.buildEnrolments(
            cohort.Id, new List<Contact>{ students[0], students[1], students[2] }, 'Confirmed');
        insert firstThree;

        Enrolment__c fourth = new Enrolment__c(
            Cohort__c = cohort.Id, Student__c = students[3].Id, Status__c = 'Confirmed');

        Test.startTest();
        Database.SaveResult result = Database.insert(fourth, false);
        Test.stopTest();

        Assert.isFalse(result.isSuccess(), 'The fourth enrolment must be rejected');
        Assert.isTrue(result.getErrors()[0].getMessage().contains('full'),
                      'The error should explain that the cohort is full');
        Assert.areEqual(3, [SELECT COUNT() FROM Enrolment__c WHERE Cohort__c = :cohort.Id],
                        'Only three enrolments should exist');
    }

    @isTest
    static void testBulkInsertRespectsCapacity() {
        Course__c course     = [SELECT Id FROM Course__c LIMIT 1];
        Cohort__c bigCohort  = TestDataFactory.createCohort(course.Id, 150, 21);
        List<Contact> students = TestDataFactory.createStudents(200);

        // Bulk loads arrive as 'Applied' (confirmation is an individual action that
        // triggers the welcome flow). Applied still consumes a seat, so the capacity
        // rule is exercised for the full 200-record batch.
        List<Enrolment__c> enrolments =
            TestDataFactory.buildEnrolments(bigCohort.Id, students, 'Applied');

        Test.startTest();
        List<Database.SaveResult> results = Database.insert(enrolments, false);
        Test.stopTest();

        Integer succeeded = 0;
        Integer blockedAsFull = 0;
        for (Database.SaveResult r : results) {
            if (r.isSuccess()) {
                succeeded++;
            } else if (r.getErrors()[0].getMessage().contains('full')) {
                blockedAsFull++;
            }
        }
        Integer committed = [SELECT COUNT() FROM Enrolment__c WHERE Cohort__c = :bigCohort.Id];

        // US-04 / NFR-01: the cohort must NEVER be oversold, even for a 200-record load.
        // (Partial-save retries plus the unique key can commit fewer than capacity in a
        // single mixed batch, but the capacity ceiling must always hold.)
        Assert.isTrue(committed <= 150, 'The cohort must never exceed its capacity of 150');
        Assert.areEqual(succeeded, committed, 'Save results must match what was committed');
        Assert.isTrue(blockedAsFull >= 50, 'At least the 50 overflow records must be blocked as full');
        Assert.isTrue(succeeded > 0, 'Records within capacity should be accepted');
    }

    @isTest
```

```
    static void testDuplicateEnrolmentBlocked() {
        Cohort__c cohort = getCohort();
        Contact student  = [SELECT Id FROM Contact LIMIT 1];

        insert new Enrolment__c(Cohort__c = cohort.Id, Student__c = student.Id, Status__c =
'Confirmed');

        Enrolment__c duplicate = new Enrolment__c(
            Cohort__c = cohort.Id, Student__c = student.Id, Status__c = 'Confirmed');

        Test.startTest();
        Database.SaveResult result = Database.insert(duplicate, false);
        Test.stopTest();

        Assert.isFalse(result.isSuccess(), 'A duplicate enrolment must be rejected');
    }

    @isTest
    static void testQueueableUpdatesContact() {
        Cohort__c cohort = getCohort();
        Contact student  = [SELECT Id FROM Contact LIMIT 1];

        Test.startTest();
        insert new Enrolment__c(Cohort__c = cohort.Id, Student__c = student.Id, Status__c =
'Confirmed');
        Test.stopTest();                                        // forces the queueable job to run

        Contact refreshed = [SELECT Total_Enrolments__c FROM Contact WHERE Id = :student.Id];
        Assert.areEqual(1, refreshed.Total_Enrolments__c, 'Contact should show one active enrolment');
    }
}
```

A.6 CohortServicesTest

```
@isTest
private class CohortServicesTest {

    @testSetup
    static void makeData() {
        Course__c course = TestDataFactory.createCourse(
            'Salesforce Developer Advanced', 'Advanced', 1500);
        TestDataFactory.createCohort(course.Id, 10, 30);
        TestDataFactory.createStudents(2);
    }

    @isTest
    static void testGetActiveCoursesAndOpenCohorts() {
        Test.startTest();
        List<Course__c> courses = CohortController.getActiveCourses();
        List<Cohort__c> cohorts = CohortController.getOpenCohorts(courses[0].Id);
        List<Cohort__c> all     = CohortController.getOpenCohorts(null);
        Test.stopTest();

        Assert.areEqual(1, courses.size(), 'One active course expected');
        Assert.areEqual(1, cohorts.size(), 'One open cohort expected for that course');
        Assert.isFalse(all.isEmpty(), 'Unfiltered query should also return cohorts');
    }

    @isTest
    static void testFindAvailableCohorts() {
        CohortAvailabilityService.Request req = new CohortAvailabilityService.Request();
        req.courseName = 'Advanced';

        Test.startTest();
        List<CohortAvailabilityService.Result> results =
```

```
            CohortAvailabilityService.findAvailableCohorts(
                new List<CohortAvailabilityService.Request>{ req });
        Test.stopTest();

        Assert.isTrue(results[0].seatsAvailable, 'Seats should be available');
        Assert.areEqual(1, results[0].cohorts.size(), 'One matching cohort expected');
        Assert.isTrue(results[0].answer.contains('Advanced'), 'Answer should name the course');
    }

    @isTest
    static void testFindAvailableCohortsNoMatch() {
        CohortAvailabilityService.Request req = new CohortAvailabilityService.Request();
        req.courseName = 'Nonexistent Course';

        Test.startTest();
        List<CohortAvailabilityService.Result> results =
            CohortAvailabilityService.findAvailableCohorts(
                new List<CohortAvailabilityService.Request>{ req });
        Test.stopTest();

        Assert.isFalse(results[0].seatsAvailable, 'No seats should be reported');
        Assert.isTrue(results[0].answer.contains('no cohorts'), 'Answer should say none were found');
    }

    @isTest
    static void testCreateEnrolmentAndDuplicateError() {
        Cohort__c cohort = [SELECT Id FROM Cohort__c LIMIT 1];
        Contact student  = [SELECT Id FROM Contact LIMIT 1];

        Test.startTest();
        Enrolment__c created = EnrolmentController.createEnrolment(cohort.Id, student.Id, 'Confirmed');

        Boolean secondAttemptFailed = false;
        try {
            EnrolmentController.createEnrolment(cohort.Id, student.Id, 'Confirmed');
        } catch (Exception e) {
            secondAttemptFailed = true;
        }
        Test.stopTest();

        Assert.isNotNull(created.Id, 'The enrolment should be created');
        Assert.isTrue(secondAttemptFailed, 'A duplicate enrolment should raise a handled exception');
    }

    @isTest
    static void testEnrolmentStatusService() {
        Cohort__c cohort = [SELECT Id FROM Cohort__c LIMIT 1];
        Contact student  = [SELECT Id, Email FROM Contact LIMIT 1];
        insert new Enrolment__c(Cohort__c = cohort.Id, Student__c = student.Id, Status__c =
'Confirmed');

        EnrolmentStatusService.Request found = new EnrolmentStatusService.Request();
        found.studentEmail = student.Email;

        EnrolmentStatusService.Request missing = new EnrolmentStatusService.Request();
        missing.studentEmail = 'nobody@oxfordable.test';

        Test.startTest();
        List<EnrolmentStatusService.Result> results = EnrolmentStatusService.checkStatus(
            new List<EnrolmentStatusService.Request>{ found, missing });
        Test.stopTest();

        Assert.isTrue(results[0].found,  'The known student should be found');
        Assert.isFalse(results[1].found, 'The unknown email should not be found');
    }
}
```

Appendix B — Lightning Web Component Source

B.1 cohortExplorer

cohortExplorer.html

```
<template>
    <lightning-card title="Open Cohorts" icon-name="standard:education">

        <div class="slds-var-m-horizontal_medium">
            <lightning-combobox
                label="Filter by course"
                placeholder="All courses"
                options={courseOptions}
                value={selectedCourseId}
                onchange={handleCourseChange}>
            </lightning-combobox>
        </div>

        <div class="slds-var-m-around_medium">
            <template lwc:if={hasCohorts}>
                <lightning-datatable
                    key-field="Id"
                    data={rows}
                    columns={columns}
                    max-row-selection="1"
                    onrowselection={handleRowSelection}>
                </lightning-datatable>
            </template>

            <template lwc:else>
                <p class="slds-text-color_weak">No open cohorts with seats available.</p>
            </template>
        </div>

    </lightning-card>
</template>
```

cohortExplorer.js

```
import { LightningElement, wire, api } from 'lwc';
import { refreshApex } from '@salesforce/apex';
import getActiveCourses from '@salesforce/apex/CohortController.getActiveCourses';
import getOpenCohorts from '@salesforce/apex/CohortController.getOpenCohorts';

const COLUMNS = [
    { label: 'Cohort',     fieldName: 'Name',               type: 'text' },
    { label: 'Course',     fieldName: 'courseName',         type: 'text' },
    { label: 'Starts',     fieldName: 'Start_Date__c',      type: 'date' },
    { label: 'Mode',       fieldName: 'Delivery_Mode__c',   type: 'text' },
    { label: 'Capacity',   fieldName: 'Capacity__c',        type: 'number',
      cellAttributes: { alignment: 'left' } },
    { label: 'Seats left', fieldName: 'Seats_Remaining__c', type: 'number',
      cellAttributes: { alignment: 'left' } }
];

export default class CohortExplorer extends LightningElement {
    columns = COLUMNS;
    selectedCourseId = null;
    courseOptions = [];
    rows = [];
    wiredCohortResult;               // held so we can refreshApex later
```

```
    @wire(getActiveCourses)
    wiredCourses({ data, error }) {
        if (data) {
            this.courseOptions = [
                { label: 'All courses', value: '' },
                ...data.map((course) => ({ label: course.Name, value: course.Id }))
            ];
        } else if (error) {
            this.courseOptions = [{ label: 'All courses', value: '' }];
        }
    }

    @wire(getOpenCohorts, { courseId: '$selectedCourseId' })
    wiredCohorts(result) {
        this.wiredCohortResult = result;
        const { data, error } = result;
        if (data) {
            this.rows = data.map((cohort) => ({
                ...cohort,
                courseName: cohort.Course__r ? cohort.Course__r.Name : ''
            }));
        } else if (error) {
            this.rows = [];
        }
    }

    get hasCohorts() {
        return this.rows && this.rows.length > 0;
    }

    handleCourseChange(event) {
        this.selectedCourseId = event.detail.value ? event.detail.value : null;
    }

    handleRowSelection(event) {
        const selected = event.detail.selectedRows;
        // Data down, events up: tell the parent which cohort was chosen.
        this.dispatchEvent(new CustomEvent('cohortselected', {
            detail: selected.length ? selected[0] : null,
            bubbles: true,
            composed: true
        }));
    }

    @api
    refresh() {
        return refreshApex(this.wiredCohortResult);
    }
}
```

cohortExplorer.js-meta.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>62.0</apiVersion>
    <isExposed>true</isExposed>
    <masterLabel>Cohort Explorer</masterLabel>
    <targets>
        <target>lightning__AppPage</target>
        <target>lightning__HomePage</target>
        <target>lightning__RecordPage</target>
    </targets>
</LightningComponentBundle>
```

B.2 enrolmentPanel

enrolmentPanel.html

```
<template>
    <lightning-card title="Enrol a Student" icon-name="standard:contact">
        <div class="slds-var-m-around_medium">

            <template lwc:if={hasCohort}>
                <p class="slds-var-m-bottom_small">
                    <strong>Cohort:</strong> {cohort.Name}  
                    <lightning-badge label={seatsLabel}></lightning-badge>
                </p>

                <lightning-record-picker
                    label="Student"
                    placeholder="Search contacts..."
                    object-api-name="Contact"
                    value={studentId}
                    onchange={handleStudentChange}>
                </lightning-record-picker>

                <div class="slds-var-m-top_medium">
                    <lightning-button
                        variant="brand"
                        label="Enrol student"
                        disabled={enrolDisabled}
                        onclick={handleEnrol}>
                    </lightning-button>
                </div>

                <template lwc:if={isWorking}>
                    <lightning-spinner alternative-text="Saving"></lightning-spinner>
                </template>
            </template>

            <template lwc:else>
                <p class="slds-text-color_weak">Select a cohort on the left to begin.</p>
            </template>

        </div>
    </lightning-card>
</template>
```

enrolmentPanel.js

```
import { LightningElement, api } from 'lwc';
import { NavigationMixin } from 'lightning/navigation';
import { ShowToastEvent } from 'lightning/platformShowToastEvent';
import createEnrolment from '@salesforce/apex/EnrolmentController.createEnrolment';

export default class EnrolmentPanel extends NavigationMixin(LightningElement) {
    @api cohort;
    studentId = null;
    isWorking = false;

    get hasCohort() {
        return !!this.cohort;
    }

    get seatsLabel() {
        return `${this.cohort.Seats_Remaining__c} seat(s) remaining`;
    }

    get enrolDisabled() {
```

```
        return !this.studentId || this.isWorking;
    }

    handleStudentChange(event) {
        this.studentId = event.detail.recordId;
    }

    async handleEnrol() {
        this.isWorking = true;
        try {
            const enrolment = await createEnrolment({
                cohortId:  this.cohort.Id,
                studentId: this.studentId,
                status:    'Confirmed'
            });

            this.showToast('Success', `Enrolment ${enrolment.Name} created.`, 'success');
            this.studentId = null;

            // Tell the page to refresh the cohort list (the seat count has changed).
            this.dispatchEvent(new CustomEvent('enrolled', {
                detail: enrolment.Id, bubbles: true, composed: true
            }));

            this[NavigationMixin.Navigate]({
                type: 'standard__recordPage',
                attributes: {
                    recordId: enrolment.Id,
                    objectApiName: 'Enrolment__c',
                    actionName: 'view'
                }
            });

        } catch (error) {
            const message = error?.body?.message || 'Something went wrong. Please try again.';
            this.showToast('Could not enrol student', message, 'error');
        } finally {
            this.isWorking = false;
        }
    }

    showToast(title, message, variant) {
        this.dispatchEvent(new ShowToastEvent({ title, message, variant }));
    }
}
```

`enrolmentPanel.js-meta.xml` — the same structure as B.1, with `<masterLabel>Enrolment Panel</masterLabel>`.

B.3 Wiring the two together — enrolmentConsole

Two ways exist, and the trade-off is a real design discussion: the **simple** wrapper we build in class (below), or independent siblings communicating over Lightning Message Service (Advanced Course).

enrolmentConsole.html

```
<template>
    <div class="slds-grid slds-gutters slds-wrap">
        <div class="slds-col slds-size_1-of-1 slds-large-size_2-of-3">
            <c-cohort-explorer oncohortselected={handleCohortSelected}></c-cohort-explorer>
        </div>
        <div class="slds-col slds-size_1-of-1 slds-large-size_1-of-3">
            <c-enrolment-panel cohort={selectedCohort} onenrolled={handleEnrolled}></c-enrolment-panel>
        </div>
    </div>
</template>
```

enrolmentConsole.js

```
import { LightningElement } from 'lwc';

export default class EnrolmentConsole extends LightningElement {
    selectedCohort;

    handleCohortSelected(event) {
        this.selectedCohort = event.detail;
    }

    handleEnrolled() {
        const explorer = this.template.querySelector('c-cohort-explorer');
        if (explorer) {
            explorer.refresh();
        }
    }
}
```

Appendix C — CLI, Manifest & Command Cheat Sheet

C.1 manifest/package.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Package xmlns="http://soap.sforce.com/2006/04/metadata">
    <types>
        <members>Course__c</members><members>Cohort__c</members>
        <members>Enrolment__c</members><members>Contact</members>
        <name>CustomObject</name>
    </types>
    <types><members>*</members><name>ApexClass</name></types>
    <types><members>EnrolmentTrigger</members><name>ApexTrigger</name></types>
    <types>
        <members>cohortExplorer</members><members>enrolmentPanel</members>
        <members>enrolmentConsole</members>
        <name>LightningComponentBundle</name>
    </types>
    <types>
        <members>Enrolment_Confirmed_Actions</members><members>Quick_Enrol_Student</members>
        <members>Cohort_Start_Reminder</members>
        <name>Flow</name>
    </types>
    <types><members>Enrolment_Console</members><name>FlexiPage</name></types>
    <types>
        <members>Course__c</members><members>Cohort__c</members>
        <members>Enrolment__c</members>
        <name>CustomTab</name>
    </types>
    <types><members>Enrolment_Console_User</members><name>PermissionSet</name></types>
    <version>62.0</version>
</Package>
```

Adjust `<version>` to your org's API version. The Extension Pack adds further types (custom metadata, labels, named credentials, Visualforce, more LWCs and flows) — deploy those with `--source-dir force-app`, which needs no manifest.

C.2 Salesforce CLI cheat sheet

PURPOSE	COMMAND
Check the CLI is installed	`sf --version`
Authorise an org / list orgs / open	`sf org login web --alias devorg` · `sf org list` · `sf org open`
Create a project	`sf project generate --name careerlaunch`
Retrieve by manifest / one item	`sf project retrieve start --manifest manifest/package.xml` · `--metadata ApexClass:CohortController`
Validate a deployment (saves nothing)	`sf project deploy start … --test-level RunLocalTests --dry-run`
Deploy / confirm	`sf project deploy start …` · `sf project deploy report`
Run all tests with coverage / one class	`sf apex run test --code-coverage --result-format human --wait 10` · `--tests EnrolmentTriggerTest`
Execute anonymous Apex	`sf apex run --file scripts/apex/seed.apex`
Assign a permission set	`sf org assign permset --name Enrolment_Console_User`

C.3 Git cheat sheet

PURPOSE	COMMAND
Start / stage / commit	`git init` · `git add .` · `git commit -m "message"`
Connect to GitHub / push	`git remote add origin https://github.com/<user>/careerlaunch.git` · `git push -u origin main`
Branch for a change	`git checkout -b feature/waitlist`
See what changed	`git status` · `git diff`
Tag a release	`git tag v1.0 && git push --tags`

Appendix D — Troubleshooting & Glossary

D.1 Troubleshooting

PROBLEM	CAUSE	FIX
LWC not available in App Builder	`isExposed` false, or wrong target	`<isExposed>true</isExposed>` + `lightning__AppPage`
App Page (e.g. Enrolment Console) missing from App Launcher	Activating an App Page does not create a Tab - they are separate steps	Setup → Tabs → **Lightning Page Tabs** → New → select the page; then grant it **Visible** in your permission set
A Flow-created email/Task subject or body is silently blank where a name should be	Two causes, often together: a plain text field with `{!...}` typed into it is never merged - only a **Text Template** resource is; and `Contact.Name` is a compound field that will not merge through a relationship even inside a Text Template	Build a Text Template resource and reference it; merge `FirstName` and `LastName` separately instead of `Name`
LWC shows “no access” errors	Missing Apex class access	Add the class to the permission set
`refreshApex` does nothing	The wired result object was not stored	Assign the *whole* wire result to a property, not just `result.data`
Agentforce menu missing in Setup	Not enabled, or unavailable in that org type	Setup → Einstein Setup → enable; else use the reference org
Agent answers vaguely / picks the wrong action	Thin topic and action descriptions	Rewrite them in plain, specific English
Queueable does not run in a test	`Test.stopTest()` missing	Async completes at stopTest
Test passes alone, fails in a run	Reliance on org data or record IDs	Create all data in `@testSetup`; never hard-code IDs
Deployment fails on coverage	Below 75%	Cover the untested branches with real assertions
Flow deployed but nothing happens	Flows can deploy inactive	Activate every flow in the target org
Emails not received from Flow	Deliverability “System email only”	Setup → Deliverability → All Email
`Too many SOQL queries: 101`	A query inside a loop	Collect IDs, then query once

D.2 Glossary — the full programme

TERM	MEANING
Agentforce	Salesforce's platform for AI agents acting on org data via topics and actions.
Apex / Trigger	Salesforce's Java-like server-side language / Apex that runs before or after saves.

Bulkification	Correct, efficient behaviour when processing many records at once.
Custom Object / External ID	A table you define (`__c`) / an outside-system identifier, unique + upsert-able.
FLS / Permission Set	Field-Level Security / an additive bundle of permissions assigned on top of a profile.
Flow	Declarative automation: record-triggered, screen, scheduled, autolaunched.
Governor limit	Per-transaction caps that keep a multi-tenant platform fair.
Invocable method	`@InvocableMethod` Apex that Flow, Agentforce and REST can call.
Lookup / Master-Detail / Roll-Up	Loose vs tight parent-child relationships; master-side count/sum fields.
LWC / Wire service	The modern UI framework / its reactive way of reading data.
Queueable Apex	An asynchronous job with fresh governor limits, enqueued with `System.enqueueJob` .
Sandbox / SOQL / DML	A copy of production for dev/test / how Apex reads / writes data.
UAT	The business confirming the solution does what was asked.

Appendix E — AI Build Prompts: the complete ordered series

Pasted into an AI coding assistant (Claude Code, Agentforce for Developers, or similar) running **inside a careerlaunch SFDX project connected to a Developer org**, these twelve prompts rebuild the entire CareerLaunch system — core and Extension Pack — and finish by seeding demo data. They are designed to be **copy-pasted one at a time, strictly in order P1 → P12**: each layer depends on the one before it. Before every prompt, a short note tells you what it will do and what you should see when it worked.

READ THIS BEFORE USING ANY PROMPT

The labs come first. You learn by typing, breaking and fixing — not by generating. Use this series for three purposes only: catching up after a session, verifying your hand-built org, or rebuilding fast in a fresh org. Whatever the AI produces: read the diff before deploying, deploy with the CLI yourself, and run the matching checkpoints. If you cannot explain a line the AI wrote, you are not finished with it. Never put real passwords or personal email addresses inside a prompt — where one is needed, the prompts use a **<placeholder>** for you to substitute.

SETUP, ONCE

1. Terminal in your `careerlaunch` project folder · 2. `sf org list` shows your org as default · 3. Start the assistant there · 4. Paste P1, review the diff, let it deploy, run the checkpoint · 5. Continue to P2, and so on.

Coverage map — every metadata type and the prompt that builds it

METADATA TYPE	PROMPT(S)
Custom objects, fields, validation rules, tabs	P1 (core) · P6 (extension + platform event)
Custom Metadata Types + records · Named Credential	P6
Custom Labels · translations	P7
Apex classes, triggers, Visualforce, tests	P2 (core) · P8 (extension)
Flows	P4 (core) · P9 (extension + automation wiring)
Lightning Web Components	P3 (console) · P10 (portal, incl. header/logout/profile)
Permission sets · FlexiPage (App Page) · **the Tab that exposes it**	P5 (core) · P10 (portal)
Experience Cloud site (clicks)	P11 checklist
Demo & test data, portal logins	P12

P1 — Core objects and fields

What this prompt does: creates the three-object data model from Section 2 — Course, Cohort and Enrolment with every field, the roll-up and formulas, both validation rules, the Contact field and the three tabs — and deploys it. **When it worked:** creating one enrolment on a capacity-3 cohort shows Enrolled Count 1 / Seats Remaining 2 with nothing typed by hand.

```
You are working in a Salesforce DX project connected to my Developer org. Create the metadata
(source format, under force-app/main/default) for the CareerLaunch data model, then deploy it.
Use these EXACT API names and settings - do not improvise names. British spelling: Enrolment, one "l".
```

```
1) Custom object Course__c (may already exist from Lab 1 - if so, verify and complete it):
   label Course/Courses, Name field = text "Course Name", enable reports and activities.
   Fields: Course_Level__c Picklist restricted (Introductory, Advanced);
   Duration_Weeks__c Number(2,0); Fee__c Currency(10,2); Active__c Checkbox default true;
   Description__c LongTextArea(1000).

2) Custom object Cohort__c: label Cohort/Cohorts, Name = AutoNumber "CO-{0000}", reports+activities.
   Fields: Course__c Lookup(Course__c) required; Start_Date__c Date required; End_Date__c Date;
   Delivery_Mode__c Picklist (Online, In-Person, Hybrid); Capacity__c Number(3,0) required default 20;
   Status__c Picklist (Planned, Open, Full, Running, Completed, Cancelled);
   Enrolled_Count__c Roll-Up Summary COUNT of Enrolment__c filtered Status__c != Cancelled;
   Seats_Remaining__c Formula Number(0dp) = Capacity__c - Enrolled_Count__c;
   Cohort_Label__c Formula Text = Course__r.Name & " - " & TEXT(Start_Date__c).
   Validation rule End_Date_After_Start_Date:
   AND(NOT(ISBLANK(End_Date__c)), End_Date__c < Start_Date__c)
   message "End Date cannot be earlier than Start Date."

3) Custom object Enrolment__c: label Enrolment/Enrolments, Name = AutoNumber "ENR-{00000}".
   Fields in this order: Cohort__c MasterDetail(Cohort__c); Student__c Lookup(Contact) required,
   relationship name Enrolments; Status__c Picklist restricted (Applied default, Confirmed,
   Waitlisted, Cancelled, Completed); Enrolment_Date__c Date; Amount_Paid__c Currency(10,2) default 0;
   Enrolment_Key__c Text(64) External ID + Unique (case-insensitive);
   Attendance_Percent__c Percent(3,0); Certificate_Issued__c Checkbox default false.
   Validation rule Payment_Not_Above_Fee: Amount_Paid__c > Cohort__r.Course__r.Fee__c
   message "Amount Paid cannot exceed the course fee."

4) On standard Contact: field Total_Enrolments__c Number(3,0), label "Total Enrolments".
5) Custom tabs for Course__c, Cohort__c and Enrolment__c (any motif).

Deploy with sf project deploy start, report any errors, list every file you created.
Do NOT create any Apex, flows, LWCs or records in this step.
```

P2 — Core Apex, trigger and tests

What this prompt does: writes the trigger + handler (capacity rule, duplicate key, defaults), the async roll-up job, the two invocable services, the two LWC controllers, and the core test classes — then runs the tests. **When it worked:** a 4th enrolment on a full cohort fails with your message, a duplicate fails on the unique key, and the test run is green.

```
In the same project (P1 deployed), create these Apex files exactly as specified by the Oxfordable
workbooks, deploy, then run all tests and show the results. API 62.0, with sharing unless stated.

1) trigger EnrolmentTrigger on Enrolment__c (before insert, before update, after insert, after update)
   - routing only: before insert -> EnrolmentTriggerHandler.onBeforeInsert(Trigger.new); before update
   -> onBeforeUpdate(Trigger.new, Trigger.oldMap); after insert/update -> onAfterChange(Trigger.new).
2) class EnrolmentTriggerHandler: setDefaults (date=today, status='Applied', amount=0 when blank);
   buildUniqueKey (Enrolment_Key__c = 15-char Cohort id + '-' + 15-char Student id);
   preventOverbooking (bulk-safe: count newly-consuming rows per cohort where Status != 'Cancelled',
   ONE query of Capacity__c/Enrolled_Count__c, allocate in memory, addError("This cohort is full.
   Please choose another cohort or add the student to the waiting list.") on overflow);
   enqueueStudentSummary (collect Student__c ids, enqueue StudentSummaryQueueable within limits).
3) class StudentSummaryQueueable implements Queueable: zero every passed Contact's
   Total_Enrolments__c, one aggregate COUNT grouped by Student__c where Status != 'Cancelled',
   update Contacts, swallow DmlException with a debug log.
4) class CohortAvailabilityService: @InvocableMethod 'Find Available Cohorts' (category
   'Oxfordable CareerLaunch'); Request.courseName; Result answer/cohorts/seatsAvailable; query Open
   future cohorts matching the name, keep Seats_Remaining__c > 0, build a readable answer.
5) class EnrolmentStatusService: @InvocableMethod 'Check Enrolment Status'; Request.studentEmail;
   Result answer/found summarising each enrolment (cohort label, start date, status).
6) class CohortController: @AuraEnabled(cacheable=true) getActiveCourses() and getOpenCohorts(Id
   courseId) - Open future cohorts, WITH SECURITY_ENFORCED, optional course filter.
7) class EnrolmentController: @AuraEnabled createEnrolment(cohortId, studentId, status) - validate,
   insert (default 'Confirmed'), catch DmlException -> AuraHandledException with a friendly message
   ("That student is already enrolled on this cohort." for duplicates), return the record with names.
8) Tests: TestDataFactory (@isTest: createCourse, createCohort, createStudents bypassing duplicate
```

```
   rules, buildEnrolments NOT inserted, createStudent(String email)); EnrolmentTriggerTest (defaults,
   overbooking negative, 200-record bulk, duplicate blocked, queueable via startTest/stopTest);
   CohortServicesTest (controllers + both services incl. error paths). 90% coverage, real assertions.
```

P3 — Core Lightning Web Components (the Enrolment Console)

What this prompt does: builds the three console components and wires them together with the data-down/events-up pattern. **When it worked:** after P5 puts them on a page, enrolling from the console shows a toast and the seat count drops without a refresh.

```
Same project; P2 deployed. Create three LWCs under force-app/main/default/lwc (html, js,
js-meta.xml each; apiVersion 62.0; isExposed true; targets lightning__AppPage, lightning__HomePage,
lightning__RecordPage). Then deploy.

1) cohortExplorer: combobox "Filter by course" fed by @wire CohortController.getActiveCourses
   (prepend "All courses"); lightning-datatable of @wire getOpenCohorts(courseId:
   '$selectedCourseId') with columns Cohort, Course (flattened Course__r.Name), Starts (date),
   Mode, Capacity, Seats left; single row selection dispatches CustomEvent 'cohortselected'
   (bubbles, composed); store the whole wire result; expose @api refresh() calling refreshApex.
2) enrolmentPanel: @api cohort; badge "<n> seat(s) remaining"; lightning-record-picker for
   Contact; Enrol button (disabled until chosen) -> imperative EnrolmentController.createEnrolment
   status 'Confirmed'; success: ShowToastEvent + dispatch 'enrolled' + NavigationMixin to the
   record; error: error.body.message in a toast, never a stack trace; spinner while working.
3) enrolmentConsole: wrapper grid - cohortExplorer (2/3) with oncohortselected setting
   selectedCohort; enrolmentPanel (1/3) with cohort={selectedCohort} and onenrolled calling
   the explorer's refresh().
```

P4 — Core flows

What this prompt does: builds the three declarative automations — confirmation actions, the Quick Enrol wizard and the scheduled reminder — as deployable flow metadata. **When it worked:** confirming an enrolment sends the email + task, and filling the last seat flips the cohort to Full by itself.

```
Same project. Create three flows as flow-meta.xml files (API 62.0, status Active) and deploy.
If any element is easier in Flow Builder, generate the closest valid metadata and tell me what to
verify there.

1) Enrolment_Confirmed_Actions - record-triggered on Enrolment__c, create-or-update, entry
   Status__c='Confirmed', only when updated to meet the condition, after-save: Get_Cohort (first
   Cohort__c where Id = $Record.Cohort__c); Send Email to $Record.Student__r.Email, subject via a
   Text Template resource "Your place is confirmed - {!$Record.Cohort__r.Cohort_Label__c}"; Create
   Task with Subject via a SEPARATE Text Template resource "Send welcome pack to
   {!$Record.Student__r.FirstName} {!$Record.Student__r.LastName}" - use FirstName/LastName, NOT
   Student__r.Name (Contact.Name is a compound field and silently renders blank when merged
   through a relationship - this is a real bug we hit and fixed), WhatId=$Record.Id,
   OwnerId=$Record.OwnerId, due today, High; Decision Seats_Remaining <= 0 -> Update cohort
   Status__c='Full'.
2) Quick_Enrol_Student - screen flow: course picklist choice set (active courses); get open cohorts
   of the choice sorted by start date; cohort radio choice (label + seats in help text); Contact
   lookup; Apex action Find Available Cohorts re-checking seats; Decision; Create Enrolment__c
   (Status 'Confirmed'); success screen with the enrolment Name; "just filled up" screen otherwise;
   every Get/Create has a Fault connector to an error screen showing $Flow.FaultMessage.
3) Cohort_Start_Reminder - schedule-triggered daily 07:00 on Enrolment__c where
   Status__c='Confirmed' and Cohort__r.Start_Date__c = today + 3 (formula); reminder email + Task.
After deploying, confirm all three show State = Active.
```

P5 — Core permission set, App Page and validation

What this prompt does: packages the core for delivery — the staff permission set (NFR-05: never edit profiles), the Enrolment Console FlexiPage **and the Tab that makes it findable**, and a validate-only deploy proving everything

ships. **When it worked:** searching `Enrolment Console` in the App Launcher returns it under *Items*, and the dry-run deploy passes with tests.

> **THE STEP ALMOST EVERYONE MISSES**
>
> **Activating an App Page does not create a Tab.** They are two separate things. An App Page with no Tab pointing at it exists in the org but has no address the App Launcher can search for — like a room with no door. This prompt builds the door as step 2 below; do not skip it.

```
Same project. Finish the core delivery layer:
1) Permission set Enrolment_Console_User (label "Enrolment Console User"): Read/Create/Edit on
   Course__c, Cohort__c, Enrolment__c; Read on Contact; field permissions on all custom fields
   (formulas and roll-ups read-only); Apex class access CohortController, EnrolmentController,
   CohortAvailabilityService, EnrolmentStatusService; flow access Quick_Enrol_Student; tab
   visibility Courses/Cohorts/Enrolments/Enrolment_Console (all four - see step 2). Deploy and
   assign it to the current user with
   sf org assign permset -n Enrolment_Console_User
2) FlexiPage Enrolment_Console ("Enrolment Console"), App Page, two regions, containing the
   enrolmentConsole LWC. THEN create force-app/main/default/tabs/Enrolment_Console.tab-meta.xml -
   a CustomTab with <flexiPage>Enrolment_Console</flexiPage> and a <motif> icon - this
   is what actually makes the page appear in the App Launcher. Deploy both together and tell me
   how to activate the page and pin it to an app's navigation bar (optional - the Tab alone is
   enough for App Launcher search).
3) Verify: run all local tests with coverage; then a validate-only deploy:
   sf project deploy start --source-dir force-app --dry-run -l RunLocalTests
4) Print the manual steps metadata cannot do: activate the app page; Setup -> Deliverability =
   All Email.
```

P6 — Extension data layer (objects, platform event, custom metadata, named credential)

What this prompt does: creates everything the Extension Pack stores or configures: the Error Log object + its publish-immediately platform event, the Invoice object with the frozen-rate design, both Custom Metadata Types with their records, the payment Named Credential, and the two new tabs. **When it worked:** a hand-made invoice shows Amount (Invoice Currency) computing itself, and marking it Paid without a reference is blocked.

```
Same project, core deployed. Create and deploy the Extension Pack data layer. Exact API names.

1) Custom object Error_Log__c: Name = AutoNumber "ERR-{00000}", reports on. Fields:
   Source_Type__c Picklist restricted (Apex default, Trigger, Flow, Integration);
   Source_Name__c Text(255); Error_Message__c LongTextArea(32768,5); Stack_Trace__c
   LongTextArea(32768,10); Record_Id__c Text(18); Severity__c Picklist restricted
   (Info, Warning, Error default, Critical). Custom tab.
2) Platform event Error_Log_Event__e, publish behavior PUBLISH IMMEDIATELY, fields:
   Source_Type__c Text(50); Source_Name__c Text(255); Error_Message__c LongTextArea(32768);
   Stack_Trace__c LongTextArea(32768); Record_Id__c Text(18); Severity__c Text(20).
3) Custom object Invoice__c: Name = AutoNumber "INV-{00000}", reports + activities. Fields in
   order: Enrolment__c MasterDetail(Enrolment__c, related list "Invoices"); Amount__c
   Currency(12,2) required; Currency_Code__c Picklist restricted (GBP default, USD, EUR);
   Exchange_Rate__c Number(12,6) default 1; Amount_In_Currency__c Formula Number(2dp) =
   Amount__c * Exchange_Rate__c (blanks as zero); Status__c Picklist restricted (Draft default,
   Sent, Paid, Cancelled); Due_Date__c Date; Paid_Date__c Date; Payment_Reference__c Text(100).
   Custom tab. Validation rule Paid_Requires_Reference:
   AND(ISPICKVAL(Status__c, "Paid"), ISBLANK(Payment_Reference__c))
   message "An invoice cannot be marked Paid without the gateway payment reference."
4) Custom Metadata Type Exchange_Rate__mdt with Rate_From_GBP__c Number(12,6) required and
   Symbol__c Text(5); records GBP=1.0/£, USD=1.27/$, EUR=1.17/€.
5) Custom Metadata Type Integration_Setting__mdt with Implementation_Class__c Text(255)
   required; record Payment_Gateway = "PaymentGatewayStub".
6) Legacy Named Credential Payment_Gateway: label "Payment Gateway",
   URL https://api.payments.example.com, Anonymous, No Authentication.
Deploy and list the files. No Apex yet.
```

P7 — Custom Labels and French translations

What this prompt does: creates every piece of user-facing text as labels (P8's certificate and P10's portal import them **by name**) plus the French translation file. Runs before the extension Apex so everything referencing a label compiles first time. **When it worked:** the labels list in Setup shows all thirteen, and the translation deploy succeeds after French is enabled.

```
Same project. Create labels/CustomLabels.labels-meta.xml with these labels (category CareerLaunch,
language en_US, protected false) - names EXACT:
Certificate_Title="Certificate of Completion" · Certificate_Awarded_To="This certificate is proudly
awarded to" · Certificate_Completion_Text="for successfully completing" · Certificate_Issued_On=
"Issued on" · Portal_My_Enrolments="My Enrolments" · Portal_My_Invoices="My Invoices" ·
Portal_My_Profile="My Profile" · Portal_Pay_Now="Pay now" · Portal_No_Records="Nothing to show
here yet." · Portal_Payment_Success="Payment received - thank you!" · Portal_Download_Certificate=
"Download certificate" · Portal_Log_Out="Log out" · Portal_Log_In="Log in"

Also create translations/fr.translation-meta.xml with the French:
Certificat de réussite · Ce certificat est fièrement décerné à · pour avoir terminé avec succès ·
Délivré le · Mes inscriptions · Mes factures · Mon profil · Payer maintenant · Rien à afficher pour
le moment. · Paiement reçu - merci ! · Télécharger le certificat · Se déconnecter · Se connecter

Deploy the labels first. Then REMIND ME to enable French (Setup -> Translation Language Settings)
before deploying the translation file - deploying translations for a disabled language fails.
```

P8 — Extension Apex, triggers and the certificate page

What this prompt does: writes the whole extension code layer: the rollback-proof error logger and its event trigger, the currency service, the complete payment stack behind the interface/factory seam, invoicing, the certificate controller + PDF page + queueable (which also **emails the PDF to the student**), the paid-invoice portal-access provisioner, the one-line trigger-handler edit, and the extension test classes. **When it worked:** the full test run is green and confirming an enrolment raises exactly one invoice.

```
Same project; P6 and P7 deployed. Create per the CareerLaunch Complete Build Guide, deploy, then
run ALL tests and show the summary. Every service that can fail must log via ErrorLogger.

Error logging: class ErrorLogger (without sharing; overloads log(e, source[, recordId]) and
log(type, source, msg, stack, recordId, severity); @InvocableMethod "Log Flow Error"
(flowName/errorMessage/recordId); every path publishes Error_Log_Event__e via EventBus, never
insert) + trigger ErrorLogEventTrigger on Error_Log_Event__e after insert copying events into
Error_Log__c.

Currency: class CurrencyService over Exchange_Rate__mdt.getInstance (no SOQL): getRate, getSymbol,
convertFromGBP (2dp half-up), format; UnknownCurrencyException.

Payments (in order): PaymentRequest (DTO: invoiceId, invoiceNumber, amount, currencyCode,
studentEmail, idempotencyKey); PaymentResult (@AuraEnabled success/transactionId/errorMessage +
ok()/fail()); interface IPaymentGateway { PaymentResult charge(PaymentRequest) };
PaymentGatewayStub (amount <= 0 fails; email containing "declined" fails; else
ok("STUB-"+invoiceNumber+"-"+currency)); PaymentGatewayFactory (reads Integration_Setting__mdt
Payment_Gateway, Type.forName, interface check, @TestVisible gatewayOverride);
PaymentGatewayHttpClient (POST callout:Payment_Gateway/v1/charges, Idempotency-Key header, 20s
timeout, CalloutException -> ErrorLogger Critical + fail; wire format in private inner classes);
InvoiceService (inherited sharing; createForConfirmedEnrolments: one Sent invoice per Confirmed
enrolment at the course fee, due +14 days, GBP rate stamped, skip fee <= 0 and enrolments with a
live non-Cancelled invoice - idempotent and bulk-safe); PaymentService (without sharing + comment
why; payInvoice: load, refuse Paid/Cancelled, idempotencyKey = invoiceId+"-"+Name, callout BEFORE
DML, on success mark Paid with reference + date and add the GBP amount to Amount_Paid__c).

Certificates: class CertificateController (?id= param -> enrolment with student/cohort/course);
Visualforce page CertificatePdf (renderAs="pdf", A4 landscape, $Label.Certificate_* merge fields);
class CertificateService (@InvocableMethod "Issue Completion Certificate"; inner IssueJob
Queueable + Database.AllowsCallouts: only Certificate_Issued__c=false, render via
Page.CertificatePdf getContent (Test.isRunningTest() -> placeholder blob), insert ContentVersion
```

```
with FirstPublishLocationId, tick the flag, AND email the student: SingleEmailMessage with
setTargetObjectId(contact), congratulations body naming the course, the PDF attached as
Messaging.EmailFileAttachment; whole job try/catch -> ErrorLogger).

Portal access on payment: class PortalAccessService (without sharing; @InvocableMethod "Grant
Portal Access" taking invoiceId; enqueues an inner Queueable because User creation is setup DML -
the mixed-DML rule: for each PAID invoice's student (must have Email + Account), if no active User
exists for that ContactId, create one on profile "Customer Community User" (username = contact
email with "@" replaced by "." + "@careerlaunch.portal", nickname/alias derived from the Contact
id), assign permission set Student_Portal_User, System.resetPassword(id, true) outside tests; then
email every student their username + the site login URL via Network.getLoginUrl; failures ->
ErrorLogger).

Trigger wiring: in EnrolmentTriggerHandler.onAfterChange, after enqueueStudentSummary, add:
InvoiceService.createForConfirmedEnrolments(newEnrolments);

Tests: ErrorLoggerTest (Test.getEventBus().deliver()), CurrencyServiceTest,
InvoicePaymentServicesTest (auto-invoice + idempotency; stub happy/declined/double-pay;
HttpCalloutMock asserting the callout: endpoint and the Idempotency-Key header; factory +
override), CertificateServiceTest, PortalAccessServiceTest (provisions user + permission set on a
paid invoice; never duplicates; ignores unpaid). Real assertions, 85%+ coverage.
```

P9 — Extension flows and automation wiring

What this prompt does: wires the extension automations: certificates on completion, portal access on payment, and error logging on every Quick Enrol fault path. **When it worked:** setting an enrolment to Completed produces the PDF + email within seconds, and paying an invoice provisions the student's login.

```
Same project; P8 deployed. Three flow changes, then deploy and confirm all are Active:
1) New record-triggered flow Issue_Certificate_On_Completion on Enrolment__c: create-or-update,
   entry Status__c='Completed', ONLY when updated to meet the condition, after-save; one Apex
   action "Issue Completion Certificate" passing enrolmentId = $Record.Id.
2) New record-triggered flow Grant_Portal_Access_On_Payment on Invoice__c: same pattern with
   entry Status__c='Paid'; one Apex action "Grant Portal Access" passing invoiceId = $Record.Id.
3) Update Quick_Enrol_Student: insert an Apex action "Log Flow Error" before the error screen
   (flowName = "Quick_Enrol_Student" literal, errorMessage = $Flow.FaultMessage) and repoint every
   faultConnector to it; its connector continues to the error screen. New active version.
Note in your summary: record-triggered flows never act retroactively - records already in the
matching state before deployment need a status nudge (away and back) to fire.
```

P10 — Portal code: controller, five LWCs, permission sets

What this prompt does: builds everything the student portal runs on — the contact-scoped controller (enrolments, invoices, profile, ownership-checked payment), the five portal components (branded header, enrolments with certificate download, invoices with Pay now, profile card, logout link), the read-only student permission set, and the staff permission-set update for invoices. **When it worked:** the automated tests prove a student sees only their own records and cannot pay someone else's invoice.

```
Same project; P8-P9 deployed. Build the portal layer and deploy:

1) Class StudentPortalController (without sharing WITH the explanatory comment: Enrolment__c is a
   master-detail child so Experience Cloud sharing sets cannot reach it; privacy = contact-scoped
   queries): resolveContactId() from UserInfo -> User.ContactId (@TestVisible contactIdOverride;
   friendly AuraHandledException when null); @AuraEnabled(cacheable=true) getMyEnrolments()
   (view wrappers: course, cohort label, status, start date, certificateIssued, certificateUrl =
   "/sfc/servlet.shepherd/document/download/"+docId found via ContentDocumentLink Title LIKE
   'Certificate%'); getMyInvoices() (number, course, amountDisplay = CurrencyService symbol +
   Amount_In_Currency + code, status, dueDate, payable = Sent/Draft); getMyProfile() (name, email,
   username, account name, member-since, count of active enrolments and of certificates);
   @AuraEnabled payInvoice(Id): COUNT() ownership check against the caller's contact BEFORE
   delegating to PaymentService; PaymentStateException -> AuraHandledException.
2) Five LWCs (isExposed, targets lightningCommunity__Page + lightningCommunity__Default; LWR has
```

```
   no SLDS so style custom components yourself; all text from the P7 labels):
   portalHeader - branded gradient hero ("CareerLaunch", tagline) with a built-in link:
   basePath+"/secur/logout.jsp" for authenticated users, basePath+"/login" for guests
   (@salesforce/community/basePath, @salesforce/user/isGuest);
   portalLogout - the same auth link as a small standalone button;
   studentEnrolments - card listing enrolments with status badges and the certificate download
   link when present;
   studentInvoices - card with amount/due/status, Pay now button -> imperative payInvoice,
   success toast with Portal_Payment_Success + the reference, refreshApex;
   studentProfile - card showing the getMyProfile() fields in a tidy grid.
3) Permission set Student_Portal_User: READ-ONLY objects + fields on Course__c, Cohort__c,
   Enrolment__c, Invoice__c; Apex class access StudentPortalController ONLY. Update
   Enrolment_Console_User: Read/Create/Edit on Invoice__c, its fields, and the Invoices +
   Error Logs tabs.
4) Add a test class covering getMyProfile and re-run StudentPortalControllerTest requirements:
   only-own-records, pay-own-invoice, reject someone else's invoice.
```

P11 — Experience Cloud site (the clicks no prompt can do)

What this prompt does: nothing in the org by itself — it makes the assistant print the exact click checklist for the parts of Experience Cloud that are configuration, not metadata. Follow the checklist by hand. **When it worked:** a private-window login as a portal user shows the branded header, both cards and the profile.

```
Print (do not attempt to execute) a numbered click checklist for standing up the CareerLaunch
portal in this org, covering exactly:
1) Setup -> Digital Experiences -> Settings: enable Digital Experiences AND tick "Allow using
   standard external profiles for self-registration, user creation, and login".
2) All Sites -> New -> Build Your Own (LWR) -> name "CareerLaunch Portal" -> Builder.
3) Delete the template placeholder content; layout: portalHeader full-width, then
   studentEnrolments and studentInvoices side by side (6/6), then studentProfile full-width.
4) Publish (also activates the site).
5) Give the running admin a role (Setup -> Roles) - portal account owners must have one.
6) Experience Builder -> Administration -> Members: add profile "Customer Community User".
7) Create a test student: Contact WITH an Account and a real email you control -> Enable
   Customer User -> profile Customer Community User -> username like
   <something-unique>@careerlaunch.portal -> assign permission set Student_Portal_User ->
   set the password from the emailed link.
8) Test in a private window: login, both cards, profile, Pay now, certificate download, Log out.
Include the two classic failure messages and their fixes: FIELD_INTEGRITY "standard external
profiles" (step 1 skipped) and "account owner must have a role" (step 5 skipped).
```

P12 — Seed demo and test data

What this prompt does: fills the org with repeatable demo data — courses, cohorts sized for the capacity demos, students (including the special `declined@` student for the failed-card demo), and optional portal logins — using an idempotent script it saves into the project. Uses placeholders, never real personal credentials. **When it worked:** the debug summary prints the counts, and re-running changes nothing.

```
Same project, everything deployed. Write scripts/apex/seed_demo.apex - an IDEMPOTENT anonymous
Apex script (safe to re-run: check-before-create on every record) that creates:
1) Courses: "Salesforce Developer - Introductory" (Introductory, 3 wks, fee 500, active) and
   "Salesforce Developer - Advanced" (Advanced, 10 wks, fee 1500, active), plus "Free Taster"
   (Introductory, 1 wk, fee 0, active).
2) Cohorts: on the Introductory course - capacity 3 starting today+14 (the overbooking demo) and
   capacity 200 starting today+30 (the bulk demo); on the Free Taster - capacity 10 today+14.
   All Online, Status Open, End Date = Start + 35.
3) An Account "Demo Students" and Contacts under it: five students with emails
   student1..student5@example.com, plus one "declined.card@example.com" (the word "declined"
   drives the payment stub's failure path). Bypass duplicate rules with DMLOptions.
4) NO enrolments or invoices - the automation creates those when we demo.
Then run it with: sf apex run -f scripts/apex/seed_demo.apex
and print the record counts. Finally, remind me (do not do it) that portal logins for demo
students are created automatically when their invoice is PAID, or manually via Enable Customer
```

```
User — and that any password I set must be typed by me in Setup, never stored in a script or
prompt.
```

HOW TO RUN THE FULL REBUILD

Fresh org, whole system: P1 → P12, one at a time, deploying and running the matching checkpoint between each. Resist merging them into one mega-prompt — layer-by-layer surfaces errors where they happen, and reviewing twelve small diffs teaches you more than approving one huge one. The Complete Build Guide remains the source of truth: its Code Pack items are the reference implementation your generated code should match.